Journal of Network and Computer Applications 242 (2025) 104218

Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

Research paper

Energy-aware elastic scaling algorithm for microservices in Kubernetes clouds

Hongjian Li [a,*], Wei Rao [a], Baojian Hu [b], Yu Tian [a], Jie Shen [a]

[a] Department of Computer Science and Technology, Chongqing University of Posts and Telecommunications, Chongqing, 400065, China
[b] China Telecom Corporation Limited Chongqing Branch, Chongqing, 400060, China

ARTICLE INFO

Keywords:
Kubernetes
Microservice
Elastic scaling
Energy consumption

ABSTRACT

Current elastic scaling algorithms are limited to the perspective of containers, and applications running within containers are treated as monolithic applications when designing scheduling algorithms. In addition, the default scaling mechanisms in Kubernetes fail to effectively distinguish and manage resource consumption of idle containers, leading to resource waste and degraded system performance. Therefore, this paper proposes an energy efficiency model based on Service Level Agreement (SLA) and an energy-aware elastic scaling algorithm based on SLA to reduce the energy consumption of microservices deployed in Kubernetes. The proposed algorithm optimizes the feedback control method in container provisioning by releasing excess container resources, and periodically runs the feedforward control algorithm and the feedback control algorithm to effectively deal with the dynamic changes of the workload. The experimental results show that the energy consumption of Kubernetes clusters in a cloud environment can be reduced by 15.34% compared with the default elastic scaling algorithms in Kubernetes and the latest algorithms.

1. Introduction

Elastic scaling algorithms have become fundamental to cloud computing, enabling dynamic resource allocation to meet fluctuating workloads. These algorithms optimize resource usage, improve performance, and reduce costs and energy consumption, which are critical for modern applications (Ruíz et al., 2022; Kjorveziroski and Filiposka, 2022; Augustyn et al., 2024). As enterprises transition from monolithic architectures to microservices, resource management faces new challenges such as inter-service dependency complexity, scaling coordination difficulties, communication and synchronization overhead, and idle-state energy consumption, necessitating optimized solutions tailored to microservices' characteristics (Schomaker et al., 2015). In Kubernetes, a widely used container orchestration platform (Tamiru et al., 2020; Carrión, 2022; Dang-Quang and Yoo, 2021), elastic scaling involves both horizontal pod autoscaler (HPA), which can increase or decrease the number of pods or nodes, and vertical pod autoscaler (VPA), which can increase or decrease the hardware resources of the pods or nodes (Ding and Huang, 2021; Herath et al., 2023; Pozdniakova et al., 2024). However, these mechanisms often fail to address the nuances of microservices architectures, leading to inefficiencies, particularly in energy consumption.

Existing research predominantly approaches container scaling with a monolithic perspective, treating applications as indivisible units. For example, Toka et al. (2020) propose a Kubernetes scaling engine based on AI-driven forecasting methods. Their approach allows cloud-tenant application providers to optimize resource allocation by balancing over-provisioning against SLA violations. Similarly, Cai and Buyya (2022) present a feedback control method combining varying-processing-rate queuing and linear models to elastically provision containers. This approach improves output error accuracy by dynamically learning reference models for different arrival rates. While these methods effectively optimize the overall application, they do not fully consider the complex dynamics of microservices architectures, particularly the challenges of dependency management and scaling coordination. The complex dependencies between microservices require that multiple factors be considered during resource scheduling, but existing scheduling strategies often fail to effectively manage these dependencies. In addition, scaling coordination is also a significant challenge. The automatic scaling mechanisms in Kubernetes typically overlook the interdependencies between services, which can lead to uncoordinated scaling and, consequently, impact the system's stability and performance.

Microservices architectures decompose applications into smaller, independent components that communicate over APIs, introducing additional overheads such as dependency management and runtime synchronization (Dragoni et al., 2017; Toka et al., 2021). Wu et al.

* Corresponding author.
E-mail address: lihj@cqupt.edu.cn (H. Li).

https://doi.org/10.1016/j.jnca.2025.104218
Received 17 October 2024; Received in revised form 16 March 2025; Accepted 13 May 2025
Available online 2 June 2025

(2022) tackle these complexities with an adaptive queuing model, which adjusts processing rates based on the ratio of synchronous calls and considers bottleneck tiers. However, even advanced methods like QAQ_JPRM (Wu et al., 2022) overlook a significant source of inefficiency in microservices: the unnecessary energy consumption caused by idle containers.

Kubernetes' default scaling mechanisms, such as the HPA and VPA, rely on metrics like CPU and memory utilization to make scaling decisions. However, these metrics fail to distinguish between active and idle containers. As a result, idle containers continue to consume resources, which significantly impacts cold start latency and overall system performance. Idle containers that are not properly scaled down cause delays in reactivation. When scaling up to handle new requests is necessary, these idle containers take time to become fully operational, leading to increased latency. Moreover, the ongoing resource consumption by idle containers can create bottlenecks that affect the performance of active workloads. Since current algorithms primarily focus on optimizing resource usage for active workloads, they do not address the energy impact of idle states. This highlights the necessity for more efficient scaling strategies that optimize resource utilization, reduce cold start latency, and enhance overall system performance.

To address the above issues, this paper proposes an energy efficiency model based on SLA and an energy-aware elastic scaling algorithm (EAES). The proposed algorithm optimizes the feedback control method in container provisioning algorithm to consider the energy consumption caused by idle containers, thereby freeing up excess container resources. The experimental results show that the energy consumption of the Kubernetes clusters in a cloud environment can reduced by 15.34% compared with Kubernetes default elastic scaling algorithms and the latest algorithms. Our research makes the following contributions:

(1) An energy-aware elastic scaling framework is designed for microservices. Under this framework, an energy efficiency model based on SLA is built for microservices, including the energy consumption of CPU while the pod is running, Power Usage Effectiveness (PUE), response time, container processing rate and its variation rate, and access arrival rate.
(2) The feedback control algorithm is improved by releasing excess containers. When the minimum number of containers required for each service calculated by the feedback control algorithm is less than the number of currently running containers, the excess container resources are released to reduce the energy consumption generated by microservices running in containers.
(3) An energy-aware elastic scaling algorithm is proposed. This algorithm uses the improved container provisioning algorithm to control the number of Pod replicas, which ensures the SLA and reduces energy consumption.

The rest of the paper is organized as follows. Section 2 describes the related works. Sections 3 and 4 introduce the proposed model and algorithm. Section 5 provides the evaluation and analysis, followed by the conclusion in Section 6.

2. Related works

Elastic scaling is an important feature of Kubernetes, as it allows automatic adjustment of resources allocated to pods based on real-time demand, thereby efficiently utilizing resources, improving performance, saving costs, and ensuring high availability of applications.

Currently, many researchers have made contributions to research on container elastic scaling algorithms. Zhong and Buyya (2020) propose an elastic instance pricing model for adjusting cluster size based on runtime workload fluctuations through multiple autoscaling algorithms. The elastic instance pricing model is designed to help container orchestration systems make appropriate decisions in dynamic environments to minimize the overall cost. The experiments demonstrate that the proposed strategy could reduce the overall cost by 23% to 32% for different types of cloud workload patterns when compared to the default Kubernetes framework Rossi et al. (2020) propose an elastic scaling mechanism called ge-kube, which extends the Kubernetes orchestration engine to enhance self-adaptation and network-aware placement capabilities. Ge-kube consists of two main components: the Elasticity Manager and the Placement Manager. The Elasticity Manager adjusts the number of containers based on workload changes and monitoring metrics, while the Placement Manager determines the placement of new application instances based on resource availability and network latency between geographically distributed worker nodes. Experimental results show that when using a network-aware placement strategy, a geographically distributed Redis cluster can handle three times more operations per second compared to Kubernetes default strategy.

Guerrero et al. (2018) propose a genetic algorithm approach, using the Non-dominated Sorting Genetic Algorithm-II, to optimize container allocation and elasticity management. Experimental results demonstrate that their approach is a suitable solution for addressing the problem of container allocation and elasticity, and it obtains better objective values than the container management policies implemented in Kubernetes. Taherizadeh et al. (2019) propose a novel dynamic multi-level (DM) autoscaling method with dynamically changing thresholds, which is tailored for container-based cloud applications. The DM method utilizes monitoring data at both infrastructure and application levels to determine when to scale up or down, with thresholds dynamically adjusted based on workload conditions. They evaluate the performance of the DM method against seven existing autoscaling methods in different synthetic and real-world workload scenarios. The experimental results indicate that the DM method has better overall performance under varied amounts of workloads than the other autoscaling methods, especially in terms of response time and the number of instantiated containers.l Toka et al. (2021) propose a Kubernetes scaling engine that utilizes machine learning forecast methods to make better automatic scaling decisions for cloud-based applications. The engine's short-term evaluation loop enables it to adapt to dynamically changing requests. They also introduce a compact management parameter for the cloud-tenant application provider to easily set their sweet spot in the resource over-provisioning vs. SLA violation trade-off. The proposed engine is evaluated using measurements of web access data in a simulation environment, showing that compared to the default baseline, it results in fewer lost requests and a slight increase in prepared resources. Balla et al. (2020) propose an adaptive autoscaler, Libra, which automatically detects the optimal resource set for a single pod, and then manages the horizontal scaling process. Additionally, if the load or the underlying virtualized environment changes, Libra adapts the resource definition for the pod and adjusts the horizontal scaling process accordingly. They validated Libra in a simulation environment and demonstrated that compared to the default Kubernetes autoscaler, it can reduce average CPU and memory utilization by up to 48% and 39%, respectively.

Toka et al. (2020) propose a Kubernetes scaling engine that makes the autoscaling decisions apt for handling the actual variability of incoming requests. In this engine, various AI-based forecast methods compete with each other via a short-term evaluation loop in order to always give the lead to the method that suits best the actual request dynamics, as soon as possible. They also introduce a compact management parameter for the cloud-tenant application provider in order to easily set their sweet spot in the resource over-provisioning vs. SLA violation trade-off. The multi-forecast scaling engine and the proposed management parameter are evaluated both in simulations and with measurements on their collected web traces to show the improved quality of fitting provisioned resources to service demand. Wu et al. (2022) propose an adaptive queuing model and queuing-length aware Jackson queuing network based method. It adjusts the processing rate of containers according to the ratio of synchronous calls and considers queuing tasks when calculating the impact of bottleneck tiers to others.

Table 1
Comparison of elastic scaling algorithms.

Study	Response time	Energy consumption	Microservice	M/M/N queue model	Adaptive processing rate	Queuing-length aware	Energy-aware
Toka et al. (2020)	✓	×	×	×	×	×	×
Cai and Buyya (2022)	✓	×	✓	✓	×	×	×
Toka et al. (2021)	✓	✓	✓	✓	×	✓	×
Wu et al. (2022)	✓	×	✓	✓	✓	✓	×
Zhong and Buyya (2020)	×	×	×	×	×	×	×
Rossi et al. (2020)	✓	✓	×	×	✓	✓	×
Guerrero et al. (2018)	×	×	×	×	×	×	×
Taherizadeh et al. (2019)	✓	×	×	×	×	×	×
Balla et al. (2020)	×	×	×	×	×	×	×
EAES	✓	✓	✓	✓	✓	✓	✓

Experiments are performed on a real Kubernetes cluster, which illustrate that the proposal obtains the lowest percentage of SLA-violations (decreasing about 6.33%–12.29%) with about 0.9% additional costs compared with existing methods of Kubernetes and other latest methods. Cai and Buyya (2022) propose a feedback control method based on a combination of varying-processing-rate queuing model and linear-model to provision containers elastically which improves the accuracy of output errors by learning reference models for different arrival rates automatically and mapping output errors from reference models to the queuing model. Their approach is compared with several state-of-art algorithms on a real Kubernetes cluster. Experimental results illustrate that their approach obtains the lowest percentage of SLA violation (decreasing no less than 8.44 percent) and the second lowest cost.

In container elastic scaling algorithms, most research work (Toka et al., 2020; Cai and Buyya, 2022) is limited to the perspective of containers or treats applications running within containers as monolithic applications when designing scheduling algorithms for container elastic scaling. They have not considered the impact of applications running within containers as microservices architecture applications (mesh multi-layered web applications) on performance and energy consumption. Only a small amount of work (Wu et al., 2022) is designed for microservices, but they overlook the fact that microservices consume the CPU even when idle, which is insufficient to reduce energy consumption.

A comparison in Table 1 is presented which illustrates the difference between the proposed EAES and the existing approaches.

3. Energy-aware elastic scaling framework

The energy-aware elastic scaling framework consists of system model (Section 3.1) and energy efficiency model based on SLA (Section 3.2).

3.1. System model

Fig. 1 illustrates the framework of the proposed EAES algorithm. It is assumed that the worker nodes in Kubernetes are located in the same data center with limited resources and a fixed number. In Kubernetes, a Pod is the basic unit of deployment. It contains one or multiple containers that run the application services specified by the user's deployment request submitted through the client.

Integrating the EAES framework into Kubernetes presented several challenges, including adapting to Kubernetes' default scaling mechanisms and ensuring seamless interaction with its existing controllers and APIs. To address these challenges, we develop an Elasticity Manager and a Scale Service to implement the EAES algorithm within Kubernetes. Metrics such as CPU utilization, request arrival rate, and response time are collected in real-time via the Microservice Monitor (an integration of the Kubernetes Metrics Server and Prometheus) and provided to the Scale Service. The Scale Service utilizes these metrics as input for the EAES algorithm to generate scaling policies. The Elasticity Manager, deployed on the Kubernetes Master Node, executes these policies by dynamically adjusting the number of Pod replicas in the cluster, thereby ensuring efficient workload management, SLA compliance, and optimized resource utilization.

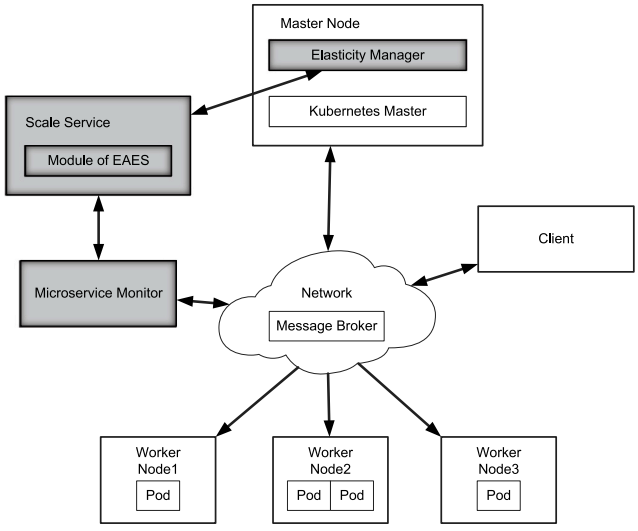


Fig. 1. The framework of the EAES algorithm.

3.2. Energy efficiency model based on SLA

Microservices invoke each other to accomplish specific business functions. Due to varying combinations of invoked microservices, different businesses typically have different mean response times (MRT).

The set of nodes is $Nodes = \{N_1, N_2, \ldots, N_s\}$, the set of pod type is $Pods = \{Pod_1, Pod_2, \ldots, Pod_n\}$, where s and n represent the number of nodes and the number of pod types, respectively. The set of microservices is $M_s = \{M_1, M_2, \ldots, M_{N_m}\}$, the set of businesses is $B_s = \{B_1, B_2, \ldots, B_{N_b}\}$, where N_m and N_b represent the number

of microservices and the number of business types, respectively. It is assumed that one Pod can only run one microservice, while one business function often requires multiple microservices to complete. Thus, business B_l is defined as $B_l = \{M_{l,1}, M_{l,2}, \dots, M_{l,N_m}\}$, where $M_{l,2}$ represents the second microservice invoked in B_l.

This paper primarily considers CPU power consumption when estimating the power consumption of physical machines because it is the main factor in server energy consumption estimation (Ruiu et al., 2017; Hogade et al., 2022). The computational power consumption of data center (Piraghaj et al., 2015) is defined as follows:

$$E_{dc} = PUE_{dc} \cdot \int_0^T \sum_{i=1}^{s} P_i(t)dt \tag{1}$$

where E_{dc} represents the power consumption of the data center, s represents the number of servers in the data center, $P_i(t)$ represents the power consumption of the ith server at time t, and PUE_{dc} represents the PUE of the data center, which is a constant.

The estimated power consumption of the ith server is defined as follows:

$$P_i(t) = \begin{cases} P_i^{idle} + \left(P_i^{max} - P_i^{idle}\right) \cdot U_{i,t} & N_m > 0 \\ 0 & N_m = 0 \end{cases} \tag{2}$$

where P_i^{idle} represents the power consumption of the ith server when idle. P_i^{max} represents the peak power consumption of the ith server. $U_{i,t}$ represents the CPU utilization of the ith server at time t, and N_m represents the number of different types of microservices.

At time t, the CPU utilization of the ith server is defined as follows:

$$U_{i,t} = \sum_{k=1}^{N_m} U_{m_{(k,i)}}(t) \tag{3}$$

where $U_{m_{(k,i)}}(t)$ represents the CPU utilization of the kth microservice on the ith server at time t, which is defined by:

$$U_{m_{(k,i)}}(t) = \sum_{j=1}^{N_{pod}} U_{pod_j}(t) \tag{4}$$

where N_{pod} represents the number of Pods corresponding to microservice $m_{(k,i)}$ and pod_j denotes the jth Pod.

In the case of multi-copy deployment, the number of copies for each type of pods is not necessarily the same. All pods are defined as follows:

$$Pod_{all} = \begin{bmatrix} Pod_{1,1} & Pod_{1,2} & \cdots & Pod_{1,k_1} \\ Pod_{2,1} & Pod_{2,2} & \cdots & Pod_{2,k_2} \\ \vdots & \vdots & \ddots & \vdots \\ Pod_{m,1} & Pod_{m,2} & \cdots & Pod_{m,k_n} \end{bmatrix} \tag{5}$$

where Pod_{m,k_n} represents the k_nth replica of the pod of type m. $k_1, k_2 \cdots k_n$ represent the number of pods for each type of microservice, and the value of k_i must satisfy $k_i \geqslant 2$.

In a Kubernetes cluster, the Round-robin(RR) approach is used when accessing multiple pods that contain microservices. To calculate network traffic between different microservices, traffic between replicas of different pods for each microservice is taken into account. Therefore, the network traffic between the ith microservice's pods and the jth microservice's pods can be expressed using the following formula:

$$NetFlowPod(Pod_i, Pod_j) = \sum_{\lambda=1}^{ki} \sum_{\varphi=1}^{kj} NetFlow(Pod_{i,\lambda}, Pod_{j,\varphi}) \tag{6}$$

where $NetFlow$ represents the network traffic between two Pods, $NetFlowPod$ denotes the network traffic between two types of microservices. Pod_i refers to the pod of the ith microservice, while $Pod_{i,\lambda}$ indicates the ith replica of the λth type of microservice. Because the network communication between pod on the same node is not forwarded by the network card, that is, the forwarding cost of the network card is not required, if $Pod_{i,\lambda}, Pod_{j,\varphi}$ on the same node, as shown in the following formula:

$$NetFlow(Pod_{i,\lambda}, Pod_{j,\varphi}) = 0 \tag{7}$$

The set of pods that communicate with Pod_i is referred to as $APod_i$, namely pod set with intimate relationship, then the total traffic $NetFlowPods(Pod_i)$ of pod of type i is defined as follows:

$$NetFlowPods(Pod_i) = \sum_{Pod_\zeta \in APod_i} NetFlowPod(Pod_i, Pod_\zeta) \tag{8}$$

All pods traffic is expressed as the following formula:

$$NetFlow_{total} = \frac{1}{2} \sum_{p \in Pods} NetFlowPods(p) \tag{9}$$

The communication energy consumption between all pods is defined as follows:

$$E_c = C \cdot NetFlow_{total} \tag{10}$$

where C represents the energy consumption coefficient of network communication.

Therefore, the total energy consumption including server-side computing energy consumption and network communication energy consumption is defined as follows:

$$E = E_{dc} + E_c \tag{11}$$

The expected response time (Wu et al., 2022) $W_{M_i}\left(N_i, \lambda_i, \hat{\mu}_i\right)$ is expressed as follows:

$$W_{M_i}\left(N_i, \lambda_i, \hat{\mu}_i\right) = P_0 \frac{\left(\lambda_i/\hat{\mu}_i\right)^{N_i}}{N_i!\left(N_i\hat{\mu}_i\right)\left(1 - \left(\frac{\lambda_i}{N_i\hat{\mu}_i}\right)^2\right)} + \frac{1}{\hat{\mu}_i} \tag{12}$$

where P_0 represents the probability that microservice M_i is not requested. N_i denotes the number of container replicas corresponding to M_i. λ_i signifies the request arrival rate for M_i and $\hat{\mu}_i$ indicates the average processing rate of M_i. P_0 is expressed as follows:

$$P_0 = \left[\sum_{z=0}^{N_i-1} \frac{1}{(z)!}\left(\frac{\lambda}{\hat{\mu}_i}\right)^z + \frac{\lambda_i^N}{N_i!\left(i - \frac{\lambda}{N_i \times \hat{\mu}_i}\right)\hat{\mu}_i^{N_i}}\right]^{-1} \tag{13}$$

where the average processing rate $\hat{\mu}_i$ of M_i is expressed by:

$$\hat{\mu}_i = \frac{1}{\Gamma_i} \tag{14}$$

where Γ_i represents the expected response time for all requests of M_i, which is expressed as follows:

$$\Gamma_i = \sum \frac{\lambda_{i,l}}{\lambda_i} \times \frac{1}{\mu_{i,l}} \left(b_l \in B_i\right) \tag{15}$$

where B_i represents the set of business types contained in M_i and $\lambda_{i,l}$ denotes the request volume for business b_l of M_i. $\mu_{i,l}$ signifies the processing rate of business b_l within M_i. b_l is one of the business types contained in M_i. The processing rate $\mu_{i,l}$ is expressed as follows:

$$\mu_{i,l} = \mu_i \times \alpha_l, b_l \in B_i \tag{16}$$

where α_l is the degradation rate of the request processing rate when the b_l issues a synchronous request.

To ensure the SLA of the microservice, the upper limit for the response time is stipulated as $\hat{W}_{M_i}$. Therefore, when the response time of microservice M_i is less than $\hat{W}_{M_i}$, the minimum number of container replicas N_i is defined by:

$$N_i = \min_{N_i \in Z^+} \left\{N_i \mid W_{M_i}\left(N_i, \lambda_i, \hat{\mu}_i\right) \leq \hat{W}_{M_i}\right\} \tag{17}$$

The proposed model ensures that during scaling events, the system adapts to workload changes by dynamically adjusting the number of replicas to meet the SLA response time requirements.

4. Elastic scaling algorithms

4.1. Scheduling algorithm design

The container resource allocation architecture of the algorithm is divided into feedforward control and feedback control.

4.1.1. Feedforward control algorithm based on workload prediction

To effectively handle workload fluctuations, predicting the workload is crucial. In this context, the multiplicative Holt–Winter model (Wu et al., 2022), which incorporates time series trends and seasonality, can accurately forecast time series data. This prediction method is primarily used to predict an increase in workload, specifically the increase in the request arrival rate λ_i for microservices in the next time interval.

Algorithm 1 The feedforward control algorithm based on workload prediction

Input: Original processing rate μ_i, decrease rate of processing rate α_l, the upper limit for the response time $\hat{W}_{M_i}$, $i \in \{1, 2, \dots, n\}$

1: **for** each microservice M_i **do**
2: Get the service arrival rate λ_i from the Microservice Monitor
3: Based on the historical data of λ_i, get the predicted arrival rate λ_i^p for M_i
4: **if** $\lambda_i > \lambda_i^p$ **then**
5: $\lambda_i \leftarrow \lambda_i^p$
6: **end if**
7: Get the real-time expected processing rate $\hat{\mu}_i$
8: Get the number of containers N_i
9: Assign N_i containers to M_i
10: **end for**

The feedforward control algorithm based on workload prediction is illustrated in Algorithm 1. Feedforward control algorithm first collects the total arrival rate λ_i for each service from the Microservice Monitor (line 1–2), then it uses the Holt–Winter model to predict the arrival rate λ_i^p for the next time interval (line 3). Using the higher arrival rate, the expected processing rate $\hat{\mu}_i$ for microservice M_i is calculated by Eq. (14) (line 4–7). Subsequently, the number of containers for the microservice is determined using $\hat{\mu}_i$ and calculated using Eq. (17) (line 8), followed by container allocation (line 9–10).

4.1.2. Queuing-length aware feedback control algorithm

During the feedforward scheduling period, due to dynamic fluctuations in workload, there may be deviations between real-time request arrival rates and predicted arrival rates. Therefore, compared to the feedforward control algorithm, the feedback control algorithm is invoked more frequently to handle workload fluctuations. When some microservices are underresourced, they are referred to as bottlenecks, and resolving these bottlenecks requires considering the impact on other services. Queuing-length aware feedback control algorithm has been widely used to model queuing network systems. This method considers the queuing request length of bottleneck services and the impact of business types on bottleneck judgment criteria before estimating the final request rate for each service.

The request arrival rate (Wu et al., 2022) for each service is expressed as follows:

$$\lambda_i' = \lambda_i + q_i + \sum_{j=1}^{n} \Delta\lambda_j \times \psi_{ji}, i \in \{1, 2, \dots, n\} \tag{18}$$

where ψ_{ji} represents the ratio of service M_i invoking M_j. Ψ changes with the variation in the proportion of different businesses in the system. Therefore, in each feedforward control cycle, the matrix Ψ can be updated according to the current business pattern, which is given as follows:

$$\Psi = \begin{pmatrix} \psi_{11} & \cdots & \psi_{1n} \\ \vdots & \psi_{ij} & \vdots \\ \psi_{n1} & \cdots & \psi_{nn} \end{pmatrix} \tag{19}$$

In Eq. (18), $\Delta\lambda_i$ represents the excess calls generated by the bottleneck tiers to other tiers (the increased request rate), which is expressed as follows:

$$\Delta\lambda_i = \begin{cases} \lambda_i - \hat{\mu}_i \times N_i + q_i & \hat{\mu}_i \times N_i < \lambda_i \\ q_i & \text{otherwise} \end{cases} \tag{20}$$

where λ_i represents the request arrival rate for M_i. $\hat{\mu}_i \times N_i$ represents the sum of the processing capacities of all containers for M_i. q_i represents the accumulated requests.

The response time upper limit $RTUL_i$ is used to determine whether service M_i is a bottleneck. It is the theoretical response time to supply sufficient resources to the service under the current workload type. Comparing the statistically derived mean response time MRT_i with $RTUL_i$ allows for more accurate identification of bottlenecks. $RTUL_i$ is expressed as follows:

$$RTUL_i = \sum_{b_l \in B_i} \frac{\lambda_{i,l}}{\lambda_i} \times T_{l,i} \tag{21}$$

where $T_{l,i}$ represents the processing time for a portion of the access path in M_i for business b_l, which is the sum of the response times of all services along the path. $T_{l,i}$ is expressed as follows:

$$T_{l,i} = \sum_{M_k \in p_{l,i}} t_k \tag{22}$$

where $p_{l,i}$ represents a partial access path within b_l that starts from M_i and extends to the final service, with $b_l \in B_i$. When there are no synchronous calls within M_i, its basic response time is t_i.

Finally, after getting the request arrival rate for each service from Eq. (17), the number of containers can be derived.

Algorithm 2 The queuing-length aware feedback control algorithm

Input: $\hat{\mu}_i$, $\hat{W}_{M_i}$, number of currently existing containers: N_i^c, $i \in \{1, 2, \dots, n\}$, Ψ, MRT, MRT_i

1: **for** microservice M_i **do**
2: Obtain λ_i, $\lambda_{i,l}$, q_i, MRT_i from log system
3: Get $RTUL_i$
4: **end for**
5: **for** microservice M_i **do**
6: **if** $MRT_i > RTUL_i$ **then**
7: Mark M_i as a bottleneck
8: Calculate $\Delta\lambda_i$
9: **end if**
10: **end for**
11: **for** microservice M_i **do**
12: Obtain the final request arrival rate λ_i'
13: Calculate the number of containers N_i
14: **if** $N_i < N_i^c$ **then**
15: $N_i \leftarrow N_i^c$
16: **end if**
17: Assign N_i containers to microservice M_i
18: **end for**

The detailed steps of the queuing-length aware feedback control algorithm are shown in Algorithm 2. Feedback control algorithm begins by iterating over each microservice, to obtain the arrival rate λ_i, $\lambda_{i,l}$, the queue length q_i in the request queue, and mean response time MRT_i from the Microservice Monitor log system (line 1–2). Then, it uses Eq. (21) to get the response time upper limit $RTUL_i$ (line 3). It iterates over each microservice again, comparing the MRT_i with the $RTUL_i$ of bottleneck tier to determine if the service is a bottleneck (line 5–7). Next, it calculates the increased request rate $\Delta\lambda_i$ by Eq. (20) (line 8). Finally, it performs one more iteration over the microservices to get the final request arrival rate λ_i' through Eq. (18) and calculates the number of containers N_i that should be allocated to the current microservice using Eq. (17) (line 11–13). If N_i is less than the current number of containers N_i^c, no action is taken. If N_i is greater than N_i^c, the number of containers corresponding to M_i is increased to N_i (line 14–18).

4.2. Improvement of the container provisioning algorithm

In the QAQ_JPRM algorithm, the feedforward control algorithm uses time series to predict the arrival rate of requests for each service and calculates the minimum number of containers required for each service by Eq. (17). In the feedback control algorithm, the queue length-aware JQN is used to estimate the arrival rate of requests for all services in the system, and Eq. (17) is applied to recalculate the minimum number of containers required for each service.

Algorithm 3 The improved queuing-length aware feedback control algorithm

Input: $\hat{\mu}_i$, $\hat{W}_{M_i}$, number of currently existing containers: N_i^c, $i \in \{1, 2, \dots, n\}$, Ψ, MRT, MRT_i, energy-saving flag $energyFlag$

1: **for** microservice M_i **do**
2: Obtain λ_i, $\lambda_{i,l}$, q_i, MRT_i from log system
3: Get $RTUL_i$
4: **end for**
5: **for** microservice M_i **do**
6: **if** $MRT_i > RTUL_i$ **then**
7: Mark M_i as a bottleneck
8: Calculate the increased arrival rate $\Delta\lambda_i$
9: **end if**
10: **end for**
11: **for** microservice M_i **do**
12: Obtain the final request arrival rate λ_i'
13: Calculate the number of containers N_i
14: **if** $N_i < N_i^c$ **then**
15: $N_{temp} \leftarrow (N_i^c - N_i)$
16: **if** $energyFlag == True$ **then**
17: Release the excess N_{temp} containers
18: **else**
19: $N_i \leftarrow N_i^c$
20: Assign N_i containers to microservice M_i
21: **end if**
22: **else**
23: Assign N_i containers to microservice M_i
24: **end if**
25: **end for**

Since the feedforward control algorithm uses the predicted arrival rate for a future time interval, while the feedback control algorithm estimates the current arrival rate based on the queue length-aware JQN, the minimum number of containers calculated by the two methods may differ. However, in the feedback control algorithm for container provisioning, when the calculated minimum number of containers is less than the current number of running containers, excess container resources are not released. This oversight ignores the CPU resource consumption caused by idle containers when there is no load, leading to additional energy consumption.

Therefore, this section considers the impact of idle contain. When the feedback control algorithm calculates that the minimum number of containers required for each service is less than the current number of running containers, the excess container resources will be released to reduce the energy consumption of microservices running in containers. To mitigate the impact of releasing excess containers on service stability and recovery time, we have implemented a gradual release process, triggered only when the system detects low usage and no impending workload surge.

The improved queuing-length aware feedback control algorithm is shown in Algorithm 3. It first iterates over each microservice, obtaining the service arrival rate λ_i, $\lambda_{i,l}$, queue length q_i, and mean response time MRT_i from the Microservice Monitor log system (line 1–2). Then, it uses Eq. (21) to determine the response time upper limit $RTUL_i$, which indicates whether the service is a bottleneck (line 3–4). In the next iteration, each microservice's MRT_i is compared to its bottleneck response time upper limit $RTUL_i$ to identify bottleneck services (line 5–7). Following this, Eq. (20) is used to calculate the increased arrival rate $\Delta\lambda_i$ (line 8–10). Finally, in another iteration over the microservices, the final request arrival rate λ_i' is obtained using Eq. (18), and the required number of containers N_i for the current microservice is calculated using Eq. (17) (line 11–13). If N_i is less than the current container number N_i^c and the energy-saving flag is True, the excess containers are released to reduce the energy consumption of the microservice (line 14–17). If N_i is greater than N_i^c, the number of containers for M_i is increased to N_i (line 22–24).

4.3. EAES algorithm

Based on the improved container provisioning algorithm and the energy efficiency model based on SLA, the basic idea of the EAES algorithm is to simultaneously run the feedforward control algorithm based on workload prediction and the improved queuing-length aware feedback control algorithm.

Algorithm 4 illustrates the specific steps of the EAES algorithm, summarized as follows:

(1) Set the energy-saving flag based on the energy consumption of the microservices when there is no load.

(2) Set the runtime cycle time for the feedforward control algorithm based on workload prediction and the improved queuing-length aware feedback control algorithm. The cycle times for these two algorithms are different, with the former having a longer cycle time.

(3) Simultaneously and periodically run these two algorithms according to their respective cycle times.

(4) When there is a change in the number of Pods calculated by both methods for a microservice, scale the number of Pods accordingly, increasing or decreasing the number of Pods.

Algorithm 4 The EAES algorithm

Input: $\hat{\mu}_i$, $\hat{W}_{M_i}$, number of currently existing containers: N_i^c, $i \in \{1, 2, \dots, n\}$, Ψ, MRT, MRT_i, μ_i, α_l

1: Calculate the energy consumption value $energyZero$ when the CPU utilization of the microservice is zero using Eq. (11)
2: After deploying the microservices, calculate the energy consumption value $energyNoWorkLoad$ using the utilization rate of the microservices with Eq. (11)
3: **if** $energyNoWorkLoad > energyZero$ **then**
4: Set $energyFlag = True$
5: **end if**
6: Set the cycle time $T1$ for Algorithm 1
7: Set the cycle time $T2$ for Algorithm 3
8: Activate Algorithm 1 and Algorithm 3 simultaneously
9: Periodically run Algorithm 1 and Algorithm 3 according to their respective cycles

Algorithm 1 performs a traversal operation over N microservices, with each step having a time complexity of O(1), so the overall time complexity of Algorithm 1 is O(N). Algorithm 3 involves three traversals, with both the container release and allocation operations having a time complexity of O(1). Therefore, the time complexity of Algorithm 3 is also O(N). Since the EAES algorithm is executed periodically by Algorithm 1 and 3, its overall time complexity is O(N).

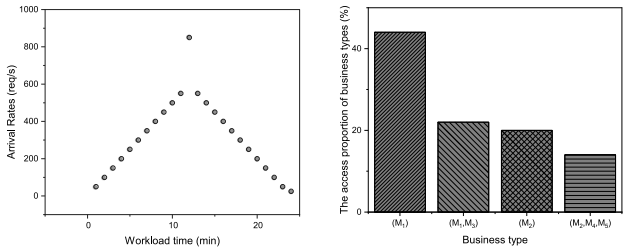
5. Performance evaluation

To compare the energy consumption and performance of the proposed algorithm (EAES) with related works, including the QAQ_JPRM algorithm (Wu et al., 2022), default elastic scaling algorithms HPA-CPU, and HPA-Request in Kubernetes, we implemented the proposed algorithm and conducted empirical evaluations through experiments deployed on a Kubernetes cloud.

QAQ_JPRM (Wu et al., 2022)—This algorithm adjusts the processing rate of containers based on the proportion of synchronous calls and

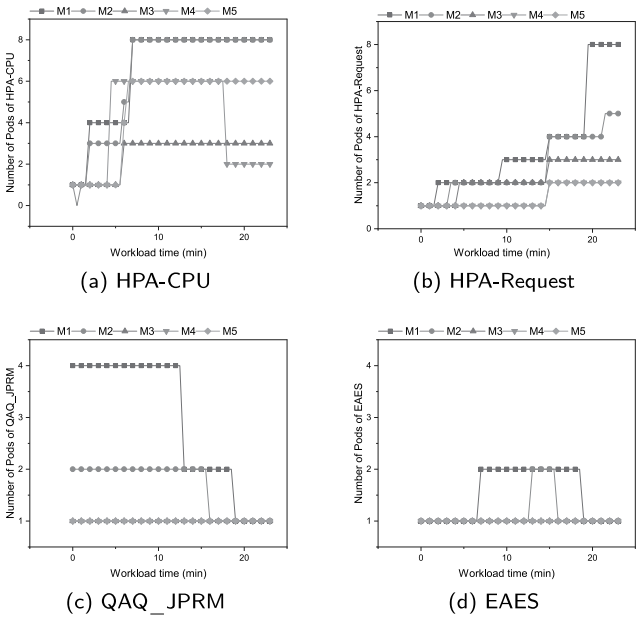
Table 2
Configuration of Kubernetes Cluster.

Node	CPU cores	Memory	Operation system	Data Center
Master	12	16GB	CentOS 6.5	Data Center 1
Node1	12	16GB	CentOS 6.5	Data Center 1
Node2	12	16GB	CentOS 6.5	Data Center 1
Node3	12	16GB	CentOS 6.5	Data Center 1
Node4	12	16GB	CentOS 6.5	Data Center 1
Node5	12	16GB	CentOS 6.5	Data Center 2



(a) The trajectory of user access arrival (b) The access proportions for different business types

Fig. 2. User access trajectory and business type access Proportions.



(a) HPA-CPU (b) HPA-Request (c) QAQ_JPRM (d) EAES

Fig. 3. The number of pods variations under different algorithms.

considers queued tasks when evaluating the impact of bottleneck tiers on other tiers.

HPA-CPU (Anon, 0000)—This algorithm performs elastic scaling based on the CPU utilization of Pods. When the CPU utilization exceeds a predefined threshold, HPA automatically increases the number of Pod replicas to handle the increased load. Conversely, when the CPU utilization falls below another threshold, HPA automatically decreases the number of Pod replicas to save resources.

HPA-Request (Anon, 0000)—This algorithm performs elastic scaling based on the resource requests of Pods, primarily focusing on CPU and memory. If the resource requests of a Pod exceed the available resources in the cluster, HPA automatically increases the number of Pod replicas. Conversely, if the resource requests are low, HPA reduces the number of Pod replicas to improve resource utilization and reduce costs.

5.1. Experimental setup

This experiment relies on a cloud platform to set up a Kubernetes cluster. The cluster consists of one Master node and five worker nodes, divided into two groups deployed in geographically separated data centers. One group contains four Nodes, while the other group contains one Node. Each virtual machine runs CentOS 6.5 with 12G CPU cores and 16G RAM, as shown in Table 2. In addition, the PUE of Data Center 2 is lower than that of Data Center 1. According to the latest comparison algorithm, the QAQ_JPRM algorithm (Wu et al., 2022), the cycle time of the feedforward control algorithm was set to 12 min, whereas the cycle time of the improved queuing-length aware feedback control algorithm was set to 3 min.

Workload: A CPU-intensive mesh network system composed of five microservices for calculating the Fibonacci sequence, named M_1, M_2, M_3, M_4, and M_5. These microservices work together in a synchronous manner, with some invoking others as needed. Finally, M_5 is responsible for writing the results of each calculation to a MySQL database.

Stress testing tool: Apache JMeter 5.5. Fig. 2(a) shows the trajectory of user access arrival rates in the workload. Fig. 2(b) depicts the access proportion of different business types of services in the web system. In Fig. 2(b), (M_1, M_3) indicates that M_1 synchronously requests M_3; (M_2, M_4, M_5) indicates that M_2 synchronously requests M_4, and M_4 synchronously requests M_5. Finally, JMeter is used to simulate user requests based on Fig. 2(a) and Fig. 2(b).

5.2. Results and analysis

Using JMeter, we observed the impact of different algorithms on the energy consumption and performance of containerized microservices. We conducted experiments three times, recording the Pod number variations, total CPU usage of the microservices, energy consumption, and SLA violation rates to validate the proposed algorithm.

5.2.1. Comparison of pod number variations

Under JMeter stress testing, variations in the number of Pods among different algorithms are shown in Fig. 3. Since HPA-CPU and HPA-Request algorithms are the default container elastic scaling algorithms supported by Kubernetes, they scale Pods up or down based on a custom threshold. When this threshold is set low, it results in frequent scaling actions, leading to excessive consumption of cluster resources. As shown in Fig. 3(a) and Fig. 3(b), HPA-CPU and HPA-Request algorithms generate a large number of Pod replicas due to their low threshold settings.

As shown in Fig. 3(c), the QAQ_JPRM algorithm calculates an appropriate number of container replicas using its model, avoiding excessive Pod scaling. This algorithm first uses a workload-based feedforward control method to calculate the expected number of Pod replicas for the next interval, using the access arrival rate at the twelfth minute, as depicted in Fig. 3(a). This results in scaling the number of Pods for microservice M_1 to 4 at the beginning. Subsequently, during the feedforward control algorithm cycle, the feedback control algorithm is executed. However, this feedback control algorithm does not shut down excess containers; it only increases the number of replicas, leading to a higher number of running container replicas and causing a waste of CPU resources.

From Fig. 3(d), it is evident that in the proposed EAES algorithm, when the feedforward control algorithm calculates the need for more container replicas, the improved feedback control algorithm promptly shuts down the excess replicas to reduce energy consumption. Simultaneously, when the workload access arrival rate increases, the algorithm scales up the containers in a timely manner. However, since the improved queuing-length aware feedback control algorithm is set to trigger every 3 min, the number of replicas at the twelfth minute, when the load rate peaks, is actually calculated based on the access arrival rate at the tenth minute.

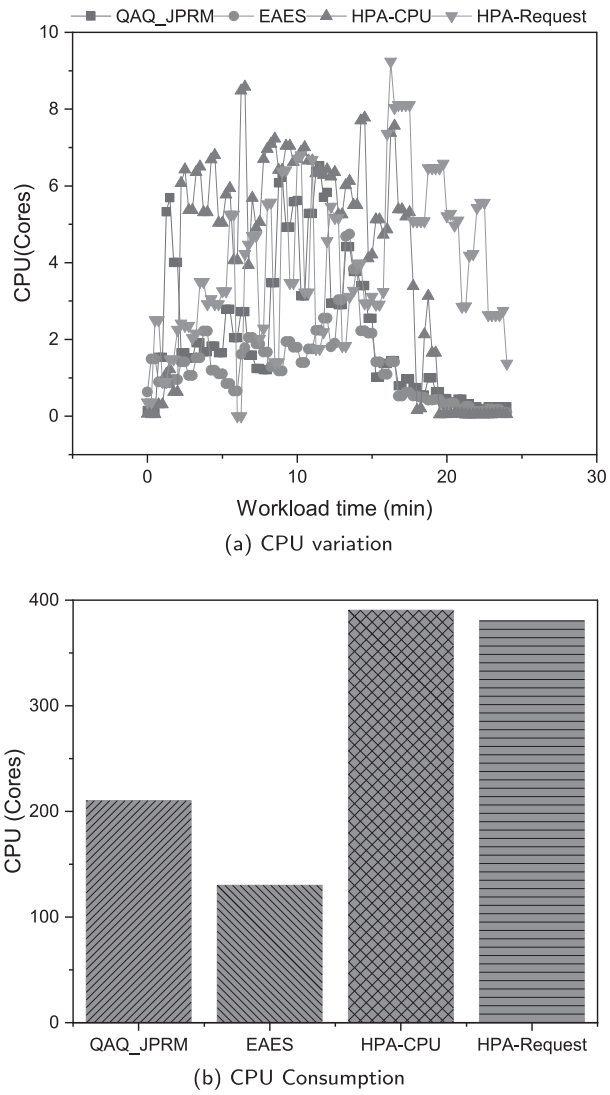


Fig. 4. CPU variation and consumption of different algorithms.

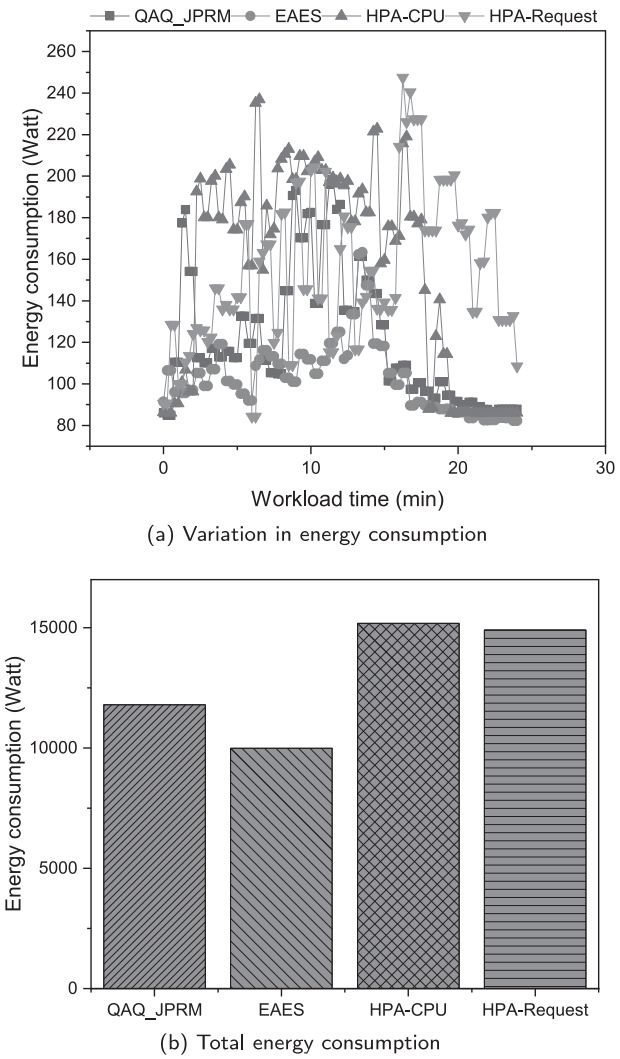


Fig. 5. Energy consumption of different algorithms.

5.2.2. Comparison of CPU consumption by microservices

Under the JMeter stress tests, the CPU consumption variations for different algorithms are illustrated in Fig. 4. Fig. 4(a) depicts the changes in CPU consumption for microservices caused by various elastic scaling algorithms under workload conditions. Fig. 4(b) shows the cumulative CPU consumption for microservices resulting from different elastic scaling algorithms, representing the total CPU consumption at different times as shown in Fig. 4(a). From Fig. 4(a), it is evident that the HPA-CPU and HPA-Request algorithms consume more CPU resources due to their low threshold settings, leading to excessive scaling operations and resource wastage. Because the workload's access arrival rate follows an increasing and then decreasing pattern, as shown in Fig. 2(a), the CPU consumption also follows a similar increasing and then decreasing trajectory.

Similarly, both the EAES and QAQ_JPRM algorithms exhibit a CPU consumption trajectory that increases and then decreases, mirroring the access arrival rate pattern shown in Fig. 2(a). The EAES algorithm considers the CPU consumption characteristics of microservices even when there is no load and reduces CPU consumption by shutting down excess containers. On the other hand, the QAQ_JPRM algorithm predicts container replicas based on peak access arrival rates, resulting in a higher number of container replicas compared to the EAES algorithm, which releases excess containers. As illustrated in Figs. 3(c) and 3(d), the QAQ_JPRM algorithm generates more containers than the EAES algorithm.

Fig. 4(b) shows the total CPU consumption for different algorithms. Ranking the four algorithms by total CPU consumption from least to greatest, the order is as follows: EAES algorithm, QAQ_JPRM algorithm, HPA-Request algorithm, and HPA-CPU algorithm.

5.2.3. Comparison of energy consumption

We conducted experiments with clusters under both different and identical PUE values. Due to experimental constraints, the PUE difference between the two data centers was small, and the cluster sizes were relatively small. As a result, the total energy consumption of the clusters under different and identical PUE values showed minimal differences under the same workload. Under JMeter stress tests, the energy consumption of different algorithms is shown in Fig. 5. Fig. 5(a) and Fig. 5(b) depict the energy consumption variations and cumulative energy consumption for microservices, with Fig. 5(b) representing the total energy consumption over time derived from Fig. 5(a). The energy consumption values are calculated using Eq. (11) based on the CPU consumption data from Fig. 4. Since energy consumption is directly related to CPU utilization, the energy consumption patterns in Fig. 5(a) align closely with the CPU consumption patterns in Fig. 4(a).

The algorithms are ranked by energy consumption from lowest to highest: EAES, QAQ_JPRM, HPA-Request, and HPA-CPU. The proposed

EAES algorithm achieves a 15.34% reduction in energy consumption compared to the latest QAQ_JPRM algorithm by optimizing CPU resource usage and minimizing unnecessary energy consumption. EAES achieves the lowest energy consumption by dynamically releasing unnecessary containers, thereby reducing CPU usage and avoiding the additional energy overhead associated with frequent scaling events, compared to the QAQ_JPRM algorithm.

The QAQ_JPRM algorithm determines the appropriate number of containers through its model. However, due to its feedforward control algorithm based on workload prediction and initial use of the peak arrival rate at the 12th minute (as shown in Fig. 2(a)), a large number of containers are created, many of which remain idle. These idle containers continue to interact with the registry center by sending heartbeat requests, consuming CPU resources and resulting in higher energy consumption.

The HPA-Request and HPA-CPU algorithms create more containers due to their lower threshold settings (as shown in Fig. 3). These containers consume CPU resources during request processing, and additionally, their heartbeat mechanisms and interactions with the registry center further increase CPU consumption. As a result, the energy consumption of the HPA-Request and HPA-CPU algorithms is higher than that of EAES and QAQ_JPRM.

In response to the energy overhead caused by frequent scaling events caused by feedback control adjustments, the EAES algorithm optimizes energy efficiency by gradually releasing excess containers. The algorithm dynamically adjusts the number of containers according to the feedforward control algorithm to ensure that containers are added or reduced only when necessary, thereby avoiding frequent scaling operations and reducing the energy fluctuations caused by them. In addition, the EAES algorithm predicts load changes in advance through feedforward control and actively adjusts resource allocation, which effectively avoids the high energy consumption caused by reactive expansion. In general, EAES minimizes the energy overhead associated with frequent scaling by precisely controlling the expansion process, showing better energy efficiency than other algorithms that may cause frequent scaling and incur higher energy overhead.

5.2.4. Comparison of SLA violation rates

To investigate the effects of different algorithms on microservice performance, this paper has quantified the response time metrics for each microservice and assessed whether these times have surpassed the response time specified in SLA. Given that the business functions of the five microservices are the same, the SLA mandates a response time of 0.05 s. This section focuses on analyzing and discussing the response times for M_1 and M_2, with the other microservices exhibiting analogous behavior.

Fig. 6 illustrate the response time variations of microservice M_1 under the control of different algorithms, compared against the dashed line representing the specified response time in the SLA, to observe the SLA violation rates. The response times of the HPA-CPU and HPA-Request algorithms mostly exceed the specified response time. This is because in resource-constrained cluster, they create numerous containers (as shown in Fig. 3), increasing intra-cluster communication, including inter-service communication and communication with the Kubernetes control plane, leading to network congestion and thereby increasing the response time and SLA violation rate of microservices. Compared to the QAQ_JPRM and EAES algorithms, their performance is poorer. The QAQ_JPRM algorithm only violates the SLA during peak workload arrival rates. Although it creates more containers in resource-constrained clusters, increasing communication with the Kubernetes control plane and affecting response time, its SLA violation rate remains lower than that of the HPA-CPU and HPA-Request algorithms because it creates fewer excess containers, resulting in lighter network congestion. The EAES algorithm, by improving the queuing-length aware feedback control algorithm and timely releasing excess containers, reduces communication with the Kubernetes control plane, effectively avoids

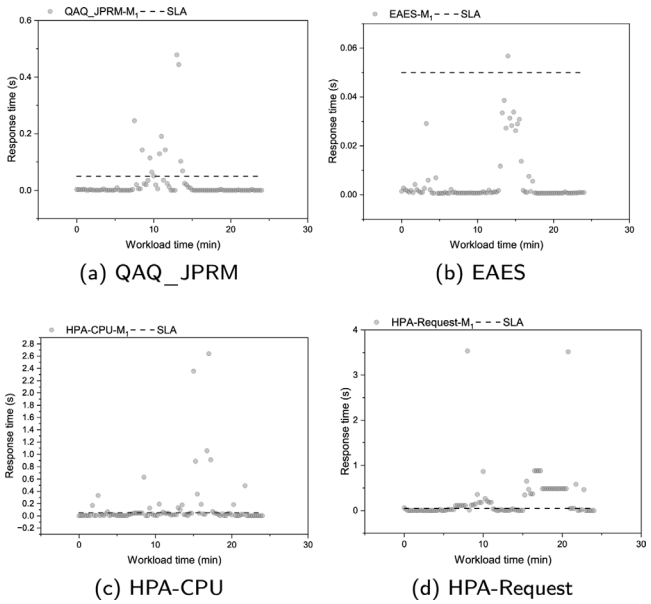


Fig. 6. Response Time of Microservice M_1.

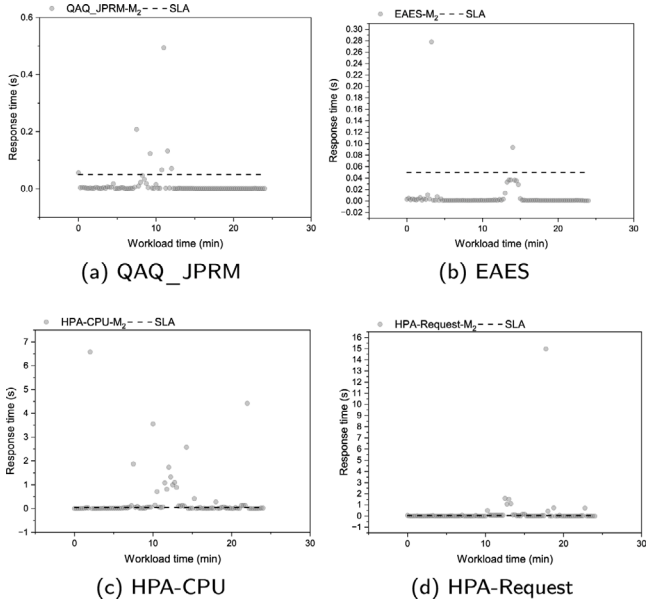


Fig. 7. Response Time of Microservice M_2.

network congestion, and thus achieves the lowest SLA violation rate, outperforming the QAQ_JPRM algorithm.

Fig. 7 demonstrate the response time variations of M_2. The QAQ_JPRM algorithm only violates the SLA during peak workload arrival rates. However, it creates more containers in resource-constrained clusters, increasing communication with the Kubernetes control plane and causing network congestion, affecting response time. So, its SLA violation rate remains higher than that of the EAES algorithm. The EAES algorithm improves the queuing-length aware feedback control algorithm, timely releases excess containers, reduces communication with the Kubernetes control plane, and calculates an appropriate replica quantity based on energy efficiency model, resulting in a lower SLA violation rate than the QAQ_JPRM algorithm. The response time variation trend of the EAES algorithm is similar to the workload arrival rate

(as shown in Fig. 2(a)). In contrast, the HPA-CPU and HPA-Request algorithms have response times that mostly exceed the specified response time in the SLA. In resource-constrained clusters, these algorithms create numerous containers, increasing intra-cluster communication, including inter-service communication and communication with the Kubernetes control plane, leading to network congestion. This results in higher microservice response times and SLA violation rates, which are higher than those of the EAES and QAQ_JPRM algorithms.

6. Conclusion

This paper proposes an energy-aware elastic scaling algorithm based on SLA. The proposed algorithm employs an improved QAQ_JPRM algorithm to control the number of Pod replicas, to ensure SLA compliance while reducing energy consumption. By integrating the CPU energy consumption of Pod runtime, PUE, network traffic, response time, container processing and access arrival rate, a novel energy efficiency model is built. Based on this model, the proposed elastic scaling algorithm minimizes the energy consumption of the Kubernetes cluster while ensuring the SLA. Experiment results show that by testing on a real Kubernetes cluster, our method saved 15.34% energy compared to existing Kubernetes studies and other recent works.

The design of EAES is based on Kubernetes, making full use of its HPA, resource monitoring mechanism and rich ecosystem to achieve energy consumption optimization and SLA guarantee. Although different container orchestration systems differ in scaling mechanism, architectural design (such as API interface, event trigger mechanism) and resource scheduling strategy, our method has certain universality in concept and mechanism and is applicable to other orchestration platforms. In addition, the optimization strategy of dynamically adjusting the number of pods has great optimization potential. By dynamically adjusting the number of pods according to service demand, resource consumption in the idle state can be effectively reduced. However, this requires finding a balance between energy efficiency, response time and service availability.

In future work, we aim to enhance the energy consumption model by integrating GPU, thereby improving its accuracy and applicability. Optimizing the heartbeat mechanism of microservices to reduce unnecessary energy consumption appears to be a viable direction, which requires further statistical analysis of the energy consumption associated with heartbeats. Moreover, we plan to evaluate our approach using more realistic business workloads while including comparisons with open-source tools such as KEDA to strengthen the representativeness and impact of our research. If conditions permit, we will explore the scalability of the proposed algorithm in large and heterogeneous Kubernetes clusters to assess its performance in diverse and complex environments.

CRediT authorship contribution statement

Hongjian Li: Software, Methodology, Conceptualization. **Wei Rao:** Writing – original draft, Data curation. **Baojian Hu:** Visualization, Investigation. **Yu Tian:** Writing – review & editing, Investigation. **Jie Shen:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Anon, 0000. Horizontal Pod Autoscaling, https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.

Augustyn, Dariusz R., Wyciślik, Łukasz, Sojka, Mateusz, 2024. Tuning a kubernetes horizontal pod autoscaler for meeting performance and load demands in cloud deployments. Appl. Sci. 14 (2), 646.

Balla, David, Simon, Csaba, Maliosz, Markosz, 2020. Adaptive scaling of kubernetes pods. In: NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium. pp. 1–5.

Cai, Zhicheng, Buyya, Rajkumar, 2022. Inverse queuing model-based feedback control for elastic container provisioning of web systems in kubernetes. IEEE Trans. Comput. 71 (2), 337–348.

Carrión, Carmen, 2022. Kubernetes as a standard container orchestrator-a bibliometric analysis. J. Grid Comput. 20 (4), 42.

Dang-Quang, Nhat-Minh, Yoo, Myungsik, 2021. Deep learning-based autoscaling using bidirectional long short-term memory for kubernetes. Appl. Sci. 11 (9), 3835.

Ding, Zhijun, Huang, Qichen, 2021. COPA: A combined autoscaling method for kubernetes. In: 2021 IEEE International Conference on Web Services. ICWS, pp. 416–425.

Dragoni, Nicola, Giallorenzo, Saverio, Lafuente, Alberto Lluch, Mazzara, Manuel, Montesi, Fabrizio, Mustafin, Ruslan, Safina, Larisa, 2017. Microservices: Yesterday, today, and tomorrow. In: Present and Ulterior Software Engineering. Springer International Publishing, Cham, pp. 195–216.

Guerrero, Carlos, Lera, Isaac, Juiz, Carlos, 2018. Genetic algorithm for multi-objective optimization of container allocation in cloud architecture. J. Grid Comput. 16, 113–135.

Herath, Isuru Pathum, Jayawardena, Samuditha, Fadhil, Ahamed, Kodagoda, Nuwan, Arachchillage, Udara Srimath S. Samaratunge, 2023. Streamlining software release process and resource management for microservice-based architecture on multi-cloud. In: 2023 25th International Multitopic Conference. INMIC, pp. 1–6.

Hogade, Ninad, Pasricha, Sudeep, Siegel, Howard Jay, 2022. Energy and network aware workload management for geographically distributed data centers. IEEE Trans. Sustain. Comput. 7 (2), 400–413.

Kjorveziroski, Vojdan, Filiposka, Sonja, 2022. Kubernetes distributions for the edge: serverless performance evaluation. J. Supercomput. 78 (11), 13728–13755.

Piraghaj, Sareh Fotuhi, Dastjerdi, Amir Vahid, Calheiros, Rodrigo N., Buyya, Rajkumar, 2015. A framework and algorithm for energy efficient container consolidation in cloud data centers. In: 2015 IEEE International Conference on Data Science and Data Intensive Systems. pp. 368–375.

Pozdniakova, Olesia, Mažeika, Dalius, Cholomskis, Aurimas, 2024. SLA-adaptive threshold adjustment for a kubernetes horizontal pod autoscaler. Electronics 13 (7), 1242.

Rossi, Fabiana, Cardellini, Valeria, Lo Presti, Francesco, Nardelli, Matteo, 2020. Geo-distributed efficient deployment of containers with kubernetes. Comput. Commun. 159, 161–174, URL https://www.sciencedirect.com/science/article/pii/S0140366419317931.

Ruiu, Pietro, Fiandrino, Claudio, Giaccone, Paolo, Bianco, Andrea, Kliazovich, Dzmitry, Bouvry, Pascal, 2017. On the energy-proportionality of data center networks. IEEE Trans. Sustain. Comput. 2 (2), 197–210.

Ruíz, Lluís Mas, Pueyo, Pere Piñol, Mateo-Fornés, Jordi, Mayoral, Jordi Vilaplana, Tehàs, Francesc Solsona, 2022. Autoscaling pods on an on-premise kubernetes infrastructure QoS-aware. IEEE Access 10, 33083–33094.

Schomaker, Gunnar, Janacek, Stefan, Schlitt, Daniel, 2015. The energy demand of data centers. In: Hilty, Lorenz M., Aebischer, Bernard (Eds.), ICT Innovations for Sustainability. Springer International Publishing, Cham, pp. 113–124.

Taherizadeh, Salman, Stankovski, Vlado, Cho, Jin-Hee, 2019. Dynamic multi-level auto-scaling rules for containerized applications. Comput. J. 62 (2), 174–197.

Tamiru, Mulugeta Ayalew, Tordsson, Johan, Elmroth, Erik, Pierre, Guillaume, 2020. An experimental evaluation of the kubernetes cluster autoscaler in the cloud. In: 2020 IEEE International Conference on Cloud Computing Technology and Science. CloudCom, pp. 17–24.

Toka, Laszlo, Dobreff, Gergely, Fodor, Balazs, Sonkoly, Balazs, 2020. Adaptive AI-based auto-scaling for kubernetes. In: 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing. CCGRID, pp. 599–608.

Toka, László, Dobreff, Gergely, Fodor, Balázs, Sonkoly, Balázs, 2021. Machine learning-based scaling management for kubernetes edge clusters. IEEE Trans. Netw. Serv. Manag. 18 (1), 958–972.

Wu, Hang, Cai, Zhicheng, Lei, Yamin, Xu, Jian, Buyya, Rajkumar, 2022. Adaptive processing rate based container provisioning for meshed micro-services in kubernetes clouds. CCF Trans. High Perform. Comput. 4 (2), 165–181.

Zhong, Zhiheng, Buyya, Rajkumar, 2020. A cost-efficient container orchestration strategy in kubernetes-based cloud computing infrastructures with heterogeneous resources. ACM Trans. Internet Technol. 20 (2).

Hongjian Li is an Associate Professor of Chongqing University of Posts and Telecommunications, China. He received his Ph.D. degree in Computer Software and Theory from the University of Electronic Science and Technology of China in 2012. His research interests include cloud computing, big data analysis, edge computing.

Wei Rao is working towards the Master degree at the School of Computer Science, Chongqing University of Posts and Telecommunications, China, under the supervision of Dr. Li. His research interests include Kubernetes and serverless computing.

Baojian Hu is Director of Big Data and AI Center of Chongqing Branch of China Telecom Co., LTD., information system project Manager, researcher of Chongqing Branch of National New Anti-Fraud Research Center, research directions include big data, privacy computing, data security, data governance, machine learning, etc.

Yu Tian is working towards the Master degree at the School of Computer Science, Chongqing University of Posts and Telecommunications, China, under the supervision of Dr. Li. His research interests include Kubernetes and serverless computing.

Jie Shen is working towards the Master degree at the School of Computer Science, Chongqing University of Posts and Telecommunications, China, under the supervision of Dr. Li. His research interests include Kubernetes and cloud computing.